RHCSA

Red Hat Certified System Administrator

Red Hat

Abdul Sami Hamedi
Cyber offensive and defensive specialist

ZOOM BYTE IT SERVICES

MAIN BRANCH

Pearson VUE Authorised Test Centre

Networking CISCO Academy

What is an Operating system?

- An operating system (OS) is system software that manages computer hardware and software resources and provides common services for computer programs
- It acts as an intermediary between users and the computer hardware. Key functions of an operating system include:
  - **Resource Management**
  - **File Management**
  - **Process Management**
  - **User Interface**
  - **Security and Access Control**

Linux History

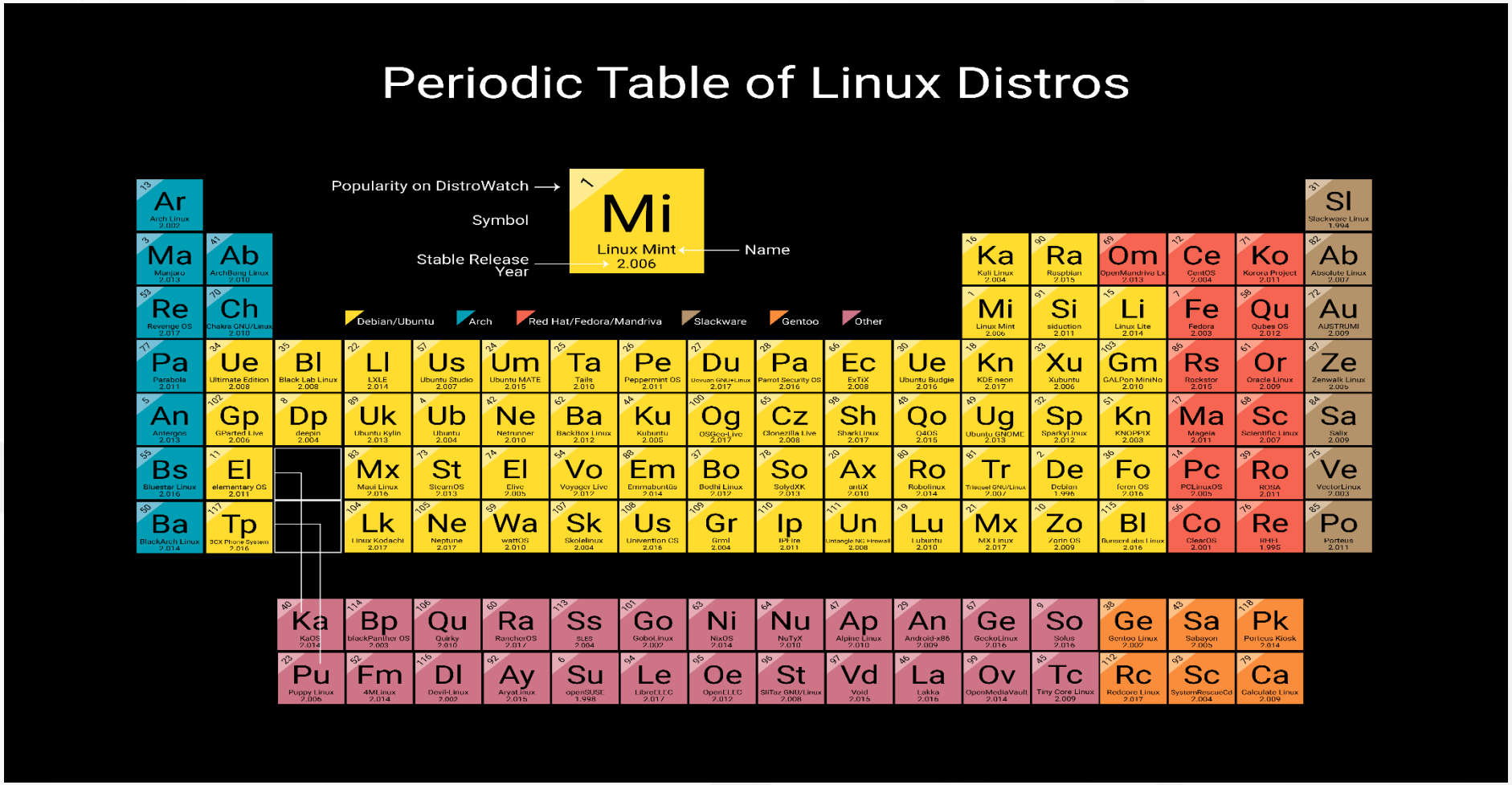
- Linux has a rich history that began in the early 1990s, inspired by Unix, which was developed in the late 1960s and early 1970s.
- In 1991, Finnish computer science student Linus Torvalds started creating Linux as a personal project, aiming to develop a free and open-source alternative to the MINIX operating system.
- Torvalds encouraged collaboration, leading to contributions from developers worldwide, which fueled rapid growth and innovation.
- Various Linux distributions, such as Slackware, Debian, and Red Hat, emerged to cater to different user needs.

Why linux?

- Choosing Linux can be beneficial for various reasons. Here are some compelling arguments for why you might consider using Linux:
  - **Cost-Effective**
  - **Open Source**
  - **Customization**
  - **Performance**
  - **Security**
  - **Community Support**
  - **Development Environment**
  - **Stability and Reliability**
  - **Privacy**
  - **More ...**

Some popular Linux distributions

- Ubuntu
- Debian
- Redhat enterprise linux
- Fedora
- CentOS
- Arch Linux
- Linux Mint
- openSUSE
- Manjaro
- Elementary OS
- Zorin OS
- Kali Linux
- Parrot OS sec



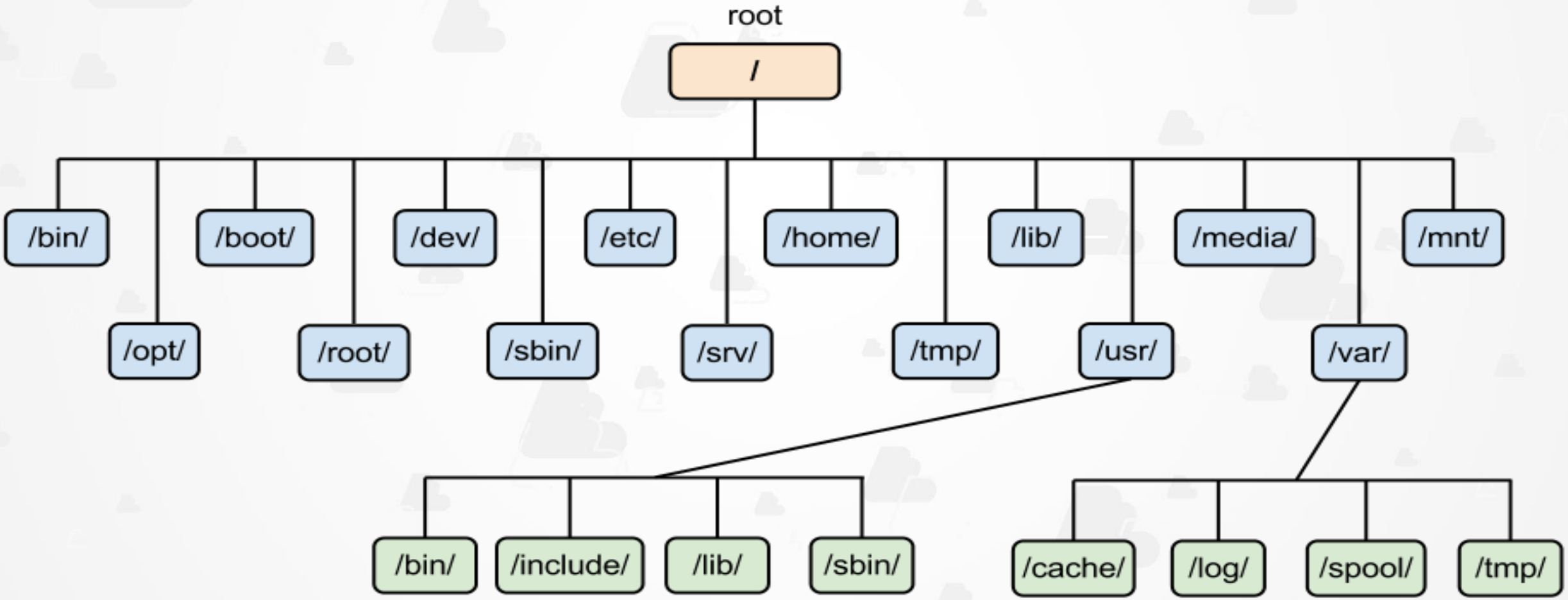
Why should I pay?

- **Reasons to Pay for Red Hat**
  - **Enterprise-Level Support**:
    - Red Hat offers 24/7 support from a team of experts who can help troubleshoot issues, provide guidance, and ensure that systems run smoothly.
  - **Security and Compliance**:
    - Red Hat provides timely security updates and patches, which are essential for maintaining the integrity and security of systems.
  - **Stability and Reliability**:
    - RHEL is known for its stability and long lifecycle, which is important for enterprises that need a reliable operating system for their applications.
  - **Access to Innovations**:
    - By paying for a subscription, organizations gain access to the latest features and innovations developed in the open-source community.
  - **Integration and Compatibility**:
    - Red Hat Enterprise Linux is designed to work seamlessly with a wide range of hardware and software solutions.
  - **Training and Resources**:
    - Subscriptions often come with access to training materials, documentation, and resources that can help teams improve their skills and knowledge about Red Hat products and open-source technologies.

What is RHCSA Exam?

- The Red Hat Certified System Administrator (RHCSA) certification, designated by the exam code EX200, is designed to validate the skills and knowledge required for system administration in Red Hat Enterprise Linux environments.
- **Exam Overview**
  - **Exam Name**: Red Hat Certified System Administrator
  - **Exam Code**: EX200
  - **Duration**: 150 minutes
  - **Number of Questions**: Approximately 20 practical tasks
  - **Passing Score**: 210 out of 300 (70%)
- **Target Audience**
  - The RHCSA certification is ideal for:
    - Experienced Red Hat Enterprise Linux system administrators.
    - Students who have completed relevant Red Hat courses.
    - IT professionals aiming to validate their skills or pursue further certifications like the Red Hat Certified Engineer (RHCE)

Linux File Hierarchy Structure



Linux Basics and Essential Tools

What is Linux Terminal?

- A **terminal** is a text-based interface that allows users to interact with the operating system by entering commands.
- **How a Terminal Works**
  - **Input**: The user types commands into the terminal.
  - **Shell**: When a command is entered, the terminal sends it to a shell, which interprets the command.
  - **Execution**: The shell performs the requested operations.
  - **Output**: After executing the command, the shell returns the output to the terminal, which displays it on the screen.

What is a shell?

- A **shell** is a command-line interface that allows users to interact with the operating system by executing commands. It serves as an intermediary between the user and the OS, translating user inputs into actions that the system can perform.
- **Types of Shells**
  - **Bash (Bourne Again SHell)**: The default shell for many Linux distributions and macOS. It features scripting capabilities and command-line editing.
  - **Zsh (Z Shell)**: An extended version of Bash with additional features like improved tab completion and customization options.
  - **Fish (Friendly Interactive SHell)**: Focused on user-friendliness and interactive use, Fish offers syntax highlighting and smart suggestions.
  - **Tcsh**: An enhanced version of the C shell (csh) with features like command line editing and programmable completions.

What is Linux Command?

- Linux commands are instructions that you can enter in a terminal or command line interface to perform specific tasks on a Linux operating system.
- **Command Structure**
  - A typical Linux command consists of three parts:
    - **Command: The name of the program or action you want to execute (e.g., ls, cp, mkdir).**
    - **Options: Modifiers that change the behavior of the command (e.g., -l, -a).**
    - **Arguments: The targets upon which the command operates (e.g., file names, directories).**
- most commands are indeed implemented as binary executable files
- **Location of Executables**
  - /bin: Essential command binaries.
  - /usr/bin: Non-essential user commands.
  - /sbin: System binaries for administrative tasks.

Working with VIM

- Vim (Vi IMproved) is a highly configurable text editor used primarily for editing source code, but it can also be employed for general text editing.
- **Vim Modes:**
  - Normal Mode:
    - Purpose: Used for navigating and manipulating text.
    - How to Enter: This is the default mode when you open Vim. You can also return to it by pressing Esc.
    - Common Commands:
      - h, j, k, l: Move left, down, up, right.
      - d: Delete text.
      - y: Yank (copy) text.
      - p: Paste text.
  - Insert Mode:
    - Purpose: Used for inserting text.
      - How to Enter: Press i (insert before the cursor) or a (append after the cursor) from Normal Mode.
      - Exiting: Press Esc to return to Normal Mode.
      - Characteristics: In this mode, you can type text as you normally would in any text editor.

Working with VIM

- **Vim Modes:**
  - Command-Line Mode:
    - Purpose: Used for executing commands.
    - How to Enter: Press : from Normal Mode.
      - Common Commands:
        - :w: Save the file.
        - :q: Quit Vim.
        - :wq: Save and quit.
  - Visual Mode:
    - Purpose: Used for selecting text.
    - How to Enter: Press v to select characters, V to select whole lines, or Ctrl+v for block selection.
    - Exiting: Press Esc to return to Normal Mode.
    - Common Actions: After selecting text, you can delete, copy, or change the selected text.

Linux Globbing

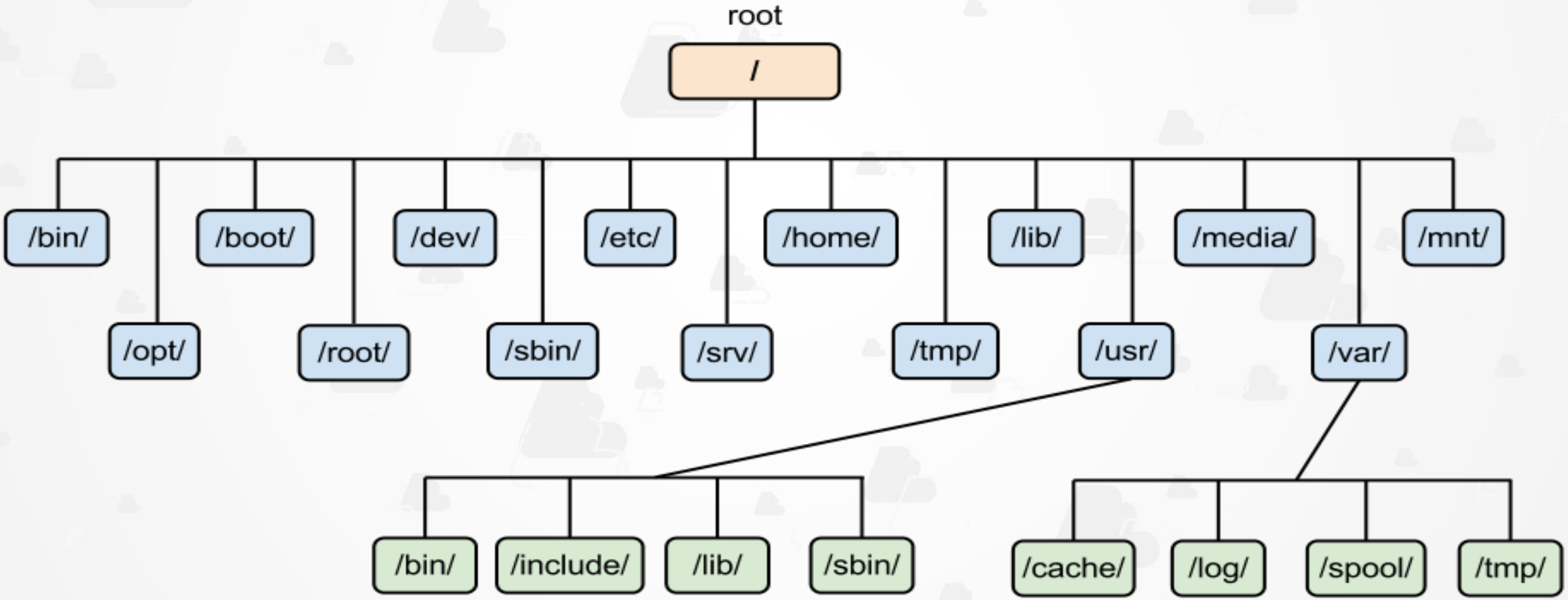
- In Linux, "globbing" refers to the process of using wildcard characters in file names or paths to match multiple files or directories.
- This is particularly useful in shell commands to perform operations on groups of files without having to specify each one individually.
- **Common Wildcard Characters**
  - * (Asterisk): Matches any number of characters, including none.
  - ? (Question Mark): Matches exactly one character.
  - [] (Square Brackets): Matches any one of the enclosed characters.
  - {} (Curly Braces): Used to specify a set of alternatives.
- **Examples of Globbing**
  - ls *.txt lists all files in the current directory that have a .txt extension.
  - ls file?.txt matches file1.txt, fileA.txt, but not file.txt or file12.txt.
  - ls file[1-3].txt matches file1.txt, file2.txt, and file3.txt.
  - echo {A,B,C}.txt produces A.txt B.txt C.txt.

Linux PIPE and IO Redirection

- **Pipe**
  - is a command in Unix-like operating systems that allows you to take the output of one command and use it as the input for another command.
  - This is a powerful feature that enables the chaining of commands to perform complex tasks efficiently.
  - ls -l | grep "txt“
- **What is IO Redirection?**
  - **I/O redirection** refers to the ability to redirect the input and output of commands to and from files or other commands.
  - This allows you to save output to a file, read input from a file, or even redirect error messages.
- **Common Redirection Operators**
  - Output Redirection (> , >>)
  - Input Redirection (<)
  - Error Redirection(2>)

Essential File Management Tools

Linux File Hierarchy Structure



Linux Essential File Management Tasks

- mkdir
  - Description: Creates a new directory.
  - mkdir new_folder
  - This command creates a directory named new_folder.
- rmdir
  - Description: Removes an empty directory.
  - rmdir empty_folder
  - This command deletes the directory empty_folder, but only if it is empty.
- cp
  - Description: Copies files or directories.
  - cp source.txt destination.txt
  - This command copies source.txt to destination.txt.
- mv
  - Description: Moves or renames files or directories.
  - mv old_name.txt new_name.txt
  - This command renames old_name.txt to new_name.txt. It can also move files to different directories.
- rm
  - Description: Removes files or directories.
  - rm file_to_delete.txt
  - This command deletes file_to_delete.txt. To remove directories and their contents, use rm -r directory_name.

File Archive and Compression

- Tar
  - **What it is**: tar stands for "tape archive." It is a utility used to combine multiple files into a single archive file.
  - **Common usage:** Often used for backup or packaging files for distribution. The resulting file typically has a .tar extension.
  - **Functionality**: While tar can combine files, it does not compress them by itself. It simply creates an archive.
- Gz
  - **What it is**: gz refers to files compressed using the GNU zip (gzip) compression algorithm.
  - **Common usage**: Files compressed with gzip usually have a .gz extension. This is often used to reduce the size of files for storage or transmission.
  - **Functionality**: Gzip compresses a single file, making it smaller. You can use gzip to compress a file, resulting in a .gz file.
- Bzip2
  - **What it is**: bzip2 is another compression utility that provides higher compression ratios than gzip, but it may be slower.
  - **Common usage**: Files compressed with bzip2 have a .bz2 extension. It's often used for compressing larger files or datasets where size reduction is critical.
  - **Functionality**: Like gzip, bzip2 compresses a single file, but with a more efficient algorithm.

Links in Linux

- **Inode**
  - an inode (index node) is a data structure that stores information about a file or a directory, except for its name and actual data content.
  - Each file or directory is represented by an inode, and the inode contains metadata about the file, such as:
    - **File type** (e.g., regular file, directory, symbolic link)
    - **Permissions** (read, write, execute permissions for owner, group, others)
    - **Owner and group information**
    - **File size**
    - **Timestamps** (e.g., last accessed, last modified, inode change times)
    - **Link count** (number of hard links pointing to the inode)
    - **Pointers to data blocks** (addresses of the data blocks on disk)
- There are two types of links in Linux: hard links and symbolic (soft) links.

Links in Linux

- **Hard Links**
  - **Hard links** are direct pointers to the inode of a file. Multiple hard links to a file mean that the file has multiple names (or paths) in the filesystem, but they all point to the same inode.
  - Deleting a hard link only removes the name from the directory. The inode and the data remain on disk as long as there is at least one hard link (or the original file) pointing to it.
  - All hard links to a file are indistinguishable and share the same inode number.
- **Symbolic (Soft) Links**
  - **Symbolic links** are separate files that contain a path to another file or directory. They are like shortcuts.
  - Symbolic links have their own inode, distinct from the target file's inode.
  - If the target file is deleted, the symbolic link becomes a "dangling" link that points to a non-existent file.

Users and Group Management

Users in linux

- In Linux, users are accounts that allow individuals or processes to access and operate the system.
- Understanding the different types of users and their roles is essential for managing a Linux environment effectively.
- **Types of Users**
  - System Users
    - These are special accounts created for running specific system services and applications, such as www-data or mysql.
    - They usually have no home directories and are not meant for direct login.
    - Their primary purpose is to manage services without exposing the system to unnecessary risk.
  - Regular Users
    - Regular users are accounts created for normal operations.
    - Each user has limited permissions compared to the root user, which enhances system security.
    - Users can only modify their files and directories, and they may have permissions to execute certain administrative tasks if granted.
    - Typically start from 1000 (on most distributions) and increment with each new user created.
  - Root User
    - The root user is the superuser in a Linux system.
    - It has unrestricted access to all commands and files.
    - **UID**: 0 (user identifier)

Users Config Files

- **/etc/default/useradd**
  - This file contains default settings for the useradd command, which is used to create new user accounts.
  - It defines parameters like the default shell, the default home directory location, and whether accounts should expire.
- **/etc/login.defs**
  - This file contains system-wide configuration settings for login-related commands, such as useradd, passwd
- **/etc/skel**
  - This directory acts as a skeleton directory for new user accounts.
- **/etc/passwd**
  - This file contains basic information about each user on the system.
- **/etc/shadow**
  - This file stores encrypted password hashes for users, along with other details like password expiration settings.
- **/etc/group**
  - This file defines groups on the system and lists which users belong to each group.

User Groups

- user groups help organize users and manage permissions, enabling fine-grained control over who can access files, directories, and system resources.
- Types of Groups
  - **Primary Group:** When a user is created, they are automatically assigned a primary group, usually with the same name as the username. Files created by the user will typically be owned by this primary group.
  - **Secondary Groups:** Users can belong to multiple secondary groups in addition to their primary one. This allows shared access to files and directories that require specific group permissions.
- Special System Groups
  - Some groups are created by default in Linux and serve system-level functions. Examples include:
    - **sudo**: Users in this group have administrative privileges and can execute commands with sudo
    - **adm**: Members can view system log files.
    - **wheel** (on some distributions): Allows administrative access.
- Managing Groups
  - Groupadd
  - usermod -aG groupname username
  - groups username
  - gpasswd -d username groupname

Permissions and Ownership

Linux File Permissions and Ownership

- In Linux, file ownership is a crucial aspect of the file system that defines which users and groups have access to files and directories.
- Every file or directory has an **owner** (usually the user who created it) and an **associated group**.
- **Key Concepts of Linux File Ownership**
  - **User Ownership**:
    - Every file is assigned to a user who is considered its owner. The owner has control over the file's permissions and can modify them.
  - **Group Ownership**:
    - Each file also belongs to a group, allowing permissions to be set for multiple users who share group membership. For instance, if multiple users belong to a group and have read or write permissions for a file, they can all access or modify it as needed.
- **Permissions**:
  - File permissions are organized in three categories: **owner**, **group**, and **others**. Each category has three basic permissions:
    - **Read (r)**: Allows viewing or reading file contents.
    - **Write (w)**: Allows modifying the file or adding/deleting contents in a directory.
    - **Execute (x)**: Allows running a file as a program or accessing a directory.

Linux Special file/folder permissions

- inux special permissions add advanced control over file access, primarily to increase security and flexibility in multi-user environments.
- There are three main special permissions in Linux: **SUID (Set User ID), SGID (Set Group ID), and the Sticky Bit**.
- SUID (Set User ID)
  - **Purpose**: Allows users to execute a file with the permissions of the file owner.
  - **How it Works**: If a file with SUID permission is executed, it temporarily grants the user the privileges of the file owner (often root) during execution.
- SGID (Set Group ID)
  - **Purpose**: Allows files to inherit group ownership, and in directories, new files inherit the parent directory's group.
  - **How it Works on Directories**: Any files created within a directory with SGID inherit the directory's group instead of the user's primary group, promoting easier group collaboration.
- Sticky Bit
  - **Purpose**: Prevents users from deleting files in shared directories unless they are the file owner.
  - **How it Works**: When applied to a directory, only the file's owner, the directory owner, or the root user can delete or rename files within it, even if other users have write access to the directory.

Linux Octal-based permissions

- In Linux, file and directory permissions are represented using a system called octal notation.
- This system allows for a compact representation of the permissions assigned to files and directories.
- Each permission is represented by a number:
  - Read is represented by 4
  - Write is represented by 2
  - Execute is represented by 1
- You can combine these values to represent different permission sets. For example:
  - Read + Write: 4 + 2 = 6
  - Read + Execute: 4 + 1 = 5
  - Write + Execute: 2 + 1 = 3
  - Read + Write + Execute: 4 + 2 + 1 = 7

Linux file/folder ACL

- ACLs allow you to specify permissions for multiple users and groups on a single file or directory.
- Setting ACLs with setfacl
  - The setfacl command is used to modify the ACL of a file or directory.
  - setfacl [options] [acl] file
  - -m: Modify the ACL by adding or changing entries.
  - -x: Remove specific ACL entries.
  - -b: Remove all ACL entries.
  - -k: Remove the default ACL.
  - -d: Set the default ACL for newly created files in a directory.
  - Granting Specific Permissions: To give user alice read and write permissions on a file named example.txt:
    - setfacl -m u:alice:rw example.txt
  - **Granting Permissions to a Group**: To give the group developers read and execute permissions on a directory:
    - setfacl -m g:developers:rx my_directory/
  - **Setting a Default ACL**: To set a default ACL for all new files created in a directory:
    - setfacl -d -m u:alice:rw my_directory/
- Retrieving ACLs with getfacl

Linux Basic Networking

Linux Basic Networking

- Linux networking is an essential area of focus for the **Red Hat Certified System Administrator (RHCSA)** exam.
- At this level, you are expected to understand and manage basic networking concepts, configure network interfaces, and troubleshoot network-related issues.
- We have 2 types of Networking configuration:
  - Runtime
  - Persistent
- Runtime Configuration
  - Runtime configuration refers to settings that are applied temporarily, taking effect only until the next system reboot, service restart, or network interface reset.
  - These configurations do not survive a reboot or restart because they aren't saved to any permanent configuration file.
  - EX: # ip addr add 192.168.1.10/24 dev <interface name>
- Persistent Configuration
  - Persistent configuration refers to settings that are saved to configuration files and survive reboots, service restarts, or interface resets.
  - Stored in configuration files that the system or service reads at boot time or when it starts up.
  - EX: Setting up a static IP in a network configuration file -- /etc/sysconfig/network-scripts/ifcfg-<interface name>

Linux Remote Management

Linux Remote Management

- Managing Linux servers remotely requires tools that offer functionality, security, and ease of use.
- SSH Tools
  - OpenSSH: The standard and widely used tool for secure remote login and command execution.
- Web-Based Management Platforms
  - Cockpit
  - Webmin
  - Ajenti
- Configuration Management Tools
  - Ansible
  - Puppet
  - Chef
- File Transfer Tools
  - SCP (Secure Copy Protocol)
  - FileZilla
- Remote Desktop Protocols and Tools
  - X2Go
  - VNC (Virtual Network Computing)
  - RDP (Remote Desktop Protocol)

SSH Authentication

- Password Authentication
  - 1: SSH Client Initialization
    - The user initiates an SSH connection using an SSH client
  - 2: Authentication Method Negotiation
    - Once the secure channel is established, the client requests a connection for the specified user .
    - The server checks its configuration (e.g., /etc/ssh/sshd_config) to determine the allowed authentication methods.
  - 3: User Provides Password
    - The user enters their password in the SSH client.
    - The password is securely sent to the server over the encrypted connection.
  - 4: Server Validates Password
    - The server checks the provided password against its user database (usually /etc/shadow on Linux).
  - 5: Access Granted or Denied
    - If the password matches, the server grants access to the user.
    - If the password is incorrect, the connection is denied or retried.

SSH Authentication

- Key-Based Authentication
  - Key-based authentication uses a cryptographic key pair (a private key and a public key) to authenticate. Here's the process:
  - 1: Key Pair Generation
    - The user generates an SSH key pair on their local machine using the **ssh-keygen** command.
      - id_rsa Private key (kept secret).
      - id_rsa.pub Public key (shared with the server).
    - 2: Copy Public Key to Server
      - The user copies their public key to the server using the **ssh-copy-id** command.
      - Alternatively, they can manually add the public key to the ~/.ssh/authorized_keys file on the server
    - 3: SSH Client Initialization
      - The user initiates an SSH connection
      - The client sends a request to the SSH server
    - 4: Server Challenges with Public Key
      - The server checks its ~/.ssh/authorized_keys file for the user and sends a challenge encrypted with the user's public key.
    - 5: Client Proves Ownership of Private Key
      - The client uses the private key to decrypt the challenge and sends back a response.
    - 6: Server Verifies
      - The server verifies the response. If it matches, the server grants access.

Linux Memory and Process Management

Linux Memory and Process Management

- The first process that starts in a Linux system is init (or its modern replacement, systemd on most contemporary distributions). This process is the parent of all other processes and has the Process ID (PID) of 1.
- What is Systemd:
  - Most modern distributions (like Ubuntu, Fedora, and Red Hat) use systemd a more advanced and feature-rich replacement for the traditional init system. It provides parallel startup and better process management.
- How Systemd works:
  - 1. Kernel Boot:
    - When the system is powered on, the bootloader (e.g., GRUB) loads the Linux kernel into memory.
    - The kernel initializes the hardware and sets up essential kernel subsystems.
  - 2. Systemd:
    - Once the kernel initialization is complete, it starts the first user-space process, typically /lib/systemd/systemd (depending on the distribution).
    - This process is responsible for initializing the rest of the system.
  - 3. System Initialization:
    - Systemd will sets up the environment by:
      - Mounting file systems.
      - Starting background services and daemons (like networking, logging, and other system services).
  - 4. Spawning Other Processes:
    - from this point, systemd spawns other processes, such as login prompts (getty), and manages all child processes.

Jobs

- Jobs are processes started by the shell e.g., bash or zsh that can run:
  - **In the foreground**: You interact with them directly.
  - **In the background**: They run without blocking your shell.
  - **Suspended**: Temporarily stopped and waiting to be resumed.
- The **jobs** command shows these processes and their current state.
- Common Job States
  - **Running**: The job is actively running in the background.
  - **Stopped**: The job is suspended, often via Ctrl+Z
  - **Terminated**: The job has completed or been killed.

Working with ps

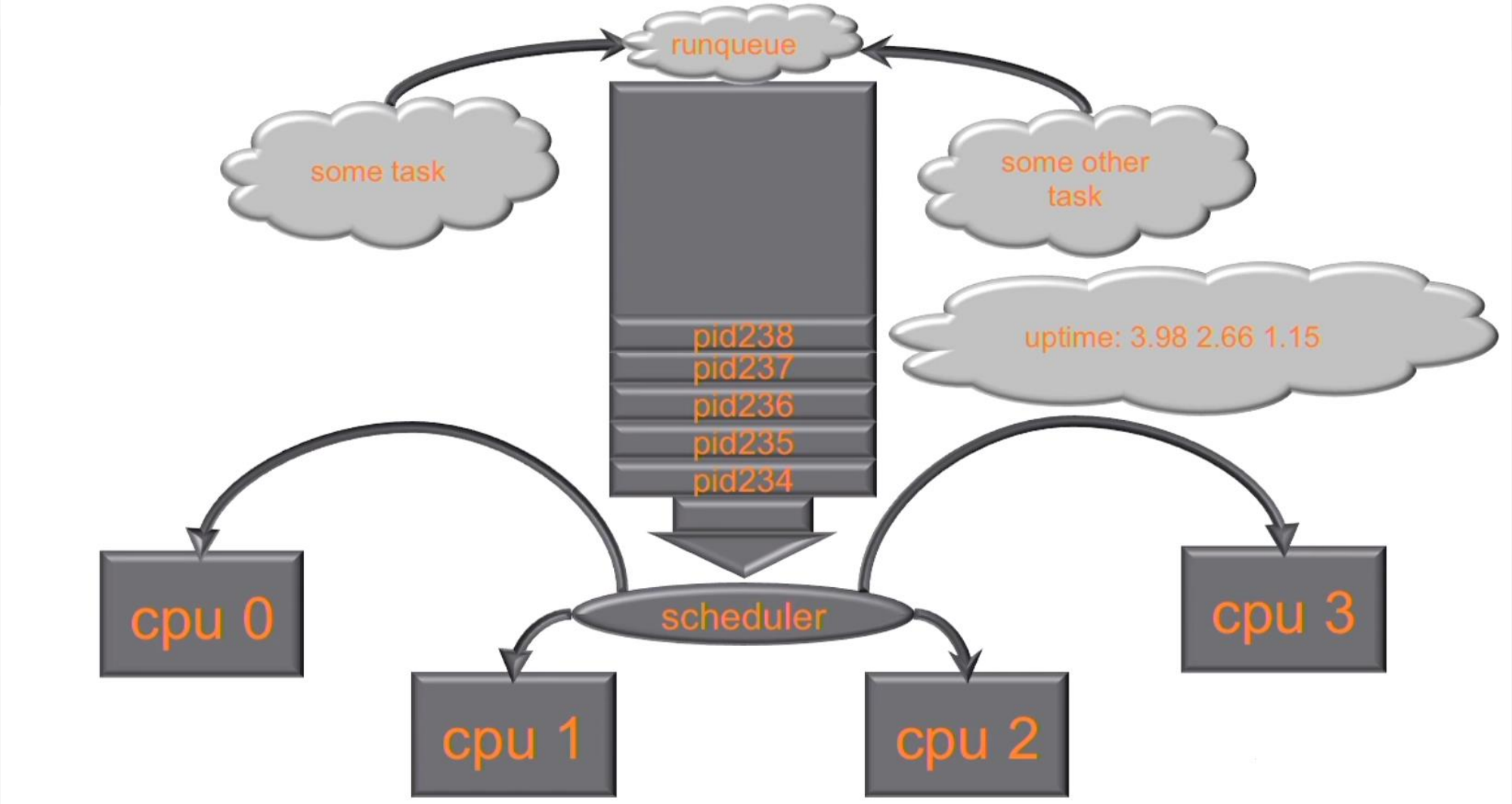
- The **ps** command in Linux is used to display information about the currently running processes on a system. It is a powerful tool for monitoring and managing processes.
- Basic syntax
  - ps aux
- Output Fields

Field	Description
PID	Process ID (unique identifier for the process).
TTY	Terminal associated with the process (or ? if none).
TIME	Total CPU time the process has used.
CMD	The command that started the process.
VSZ	Virtual memory size used by the process (in kilobytes).
RSS	Resident Set Size; non-swappable physical memory used by the process (in kilobytes).
PPID	Parent Process ID (process that spawned this one).

Working with ps cont…

- Process State
  - Here are some common process state codes:
    - **R**: Running.
    - **S**: Sleeping (idle, waiting for an event).
    - **T**: Stopped or traced.
    - **Z**: Zombie (terminated but not cleaned up by its parent).
    - **Ss**: Sleeping (idle) with a session leader.
    - **I:** Idle kernel thread (specific to kernel threads).
    - **<:** The process has a high scheduling priority.
    - **N:** The process has a low scheduling priority.
    - **+:** The process is in the foreground process group associated with a terminal.

Understanding Performance Load



Working with ps cont...

Signal	Number	Description	Default Action
SIGHUP	1	Hangup: Reload configuration or terminate the process if not caught.	Terminate
SIGINT	2	Interrupt: Sent when a user presses Ctrl+C.	Terminate
SIGQUIT	3	Quit: Sent when a user presses Ctrl+\ (generates a core dump).	Terminate and dump core
SIGKILL	9	Kill: Immediately terminate the process. Cannot be caught or ignored.	Terminate
SIGTERM	15	Terminate: Graceful termination, allowing the process to clean up.	Terminate
SIGSTOP	19 (or 17/23, system-dependent)	Stop: Suspend the process. Cannot be caught or ignored.	Stop process
SIGCONT	18 (or 19/25, system-dependent)	Continue: Resume a suspended process.	Continue execution

Real-Time Processes and Normal Processes in Linux

- Real-Time Processes
  - Have the highest priority
  - Always preempt normal processes, meaning they are guaranteed CPU time if they need it
- Normal Processes
  - Lower priority compared to real-time processes. Nice values determine their effective priority (-20 to 19).
  - Suitable for general-purpose applications like web browsing, text editing, or background tasks
- What is the Nice Value?
  - The nice value is a user-space mechanism for influencing the priority of normal processeas. It adjusts the weight given to a process in the CPU scheduling queue.
  - -20 (highest priority) to 19 (lowest priority).

What is Swap memory?

- Swap is a space on a disk that is used as virtual memory when the system's physical RAM is fully utilized. It acts as an overflow area for storing temporarily inactive pages of memory, allowing the system to handle more tasks than the available RAM alone.
- Benefits of Swap:
  - Prevents the system from crashing when memory is low.
  - Supports hibernation, as the system state can be saved to swap.
- The swappiness parameter in Linux determines how aggressively the kernel uses swap space. You can change it temporarily or permanently.
  - /proc/sys/vm/swappiness
  - Understanding Swappiness Values
    - **0**: Avoid using swap unless absolutely necessary.
    - **1–59**: Use swap sparingly, prioritizing RAM usage.
    - **60 (Default)**: Balanced use of swap and RAM.
    - **100**: Use swap space aggressively.

Software Management in Redhat

What is Package manager?

- is a software tool designed to simplify the process of installing, updating, configuring, and managing software packages on a system.
- It automates tasks that would otherwise require manual downloading, installation, and dependency resolution.
- Types of Package Managers
  - Low-Level Package Managers
    - These work directly with individual packages (e.g., .rpm, .deb) but require manual dependency management.
    - RPM (Red Hat Package Manager)
    - DPKG (Debian Package)
  - High-Level Package Managers
    - These provide more user-friendly features, such as automatic dependency resolution.
    - YUM (Yellowdog Updater Modified)
    - DNF (Dandified YUM)
    - APT (Advanced Package Tool)

What is Package manager cont…

- Examples of Common Linux Package Managers
  - **RPM/YUM/DNF** (Red Hat-based distributions like RHEL, CentOS, Fedora)
  - **APT/Dpkg** (Debian-based distributions like Ubuntu)
  - **Pacman** (Arch Linux)
  - **Zypper** (openSUSE)
  - **Flatpak** and **Snap**

What is repository?

- (often abbreviated as "repo") is a centralized storage location for software packages.
- In the context of Linux systems like Red Hat-based distributions, a repository contains pre-built software packages (e.g., .rpm files) along with metadata that allows package managers like YUM or DNF to locate, download, and install them.
- Types of Repositories
  - Official Repositories
    - Maintained by the operating system vendor (e.g., Red Hat or CentOS).
    - Contain stable and tested software packages.
  - Third-Party Repositories
    - Maintained by external organizations or developers.
    - May provide additional or specialized software not included in official repositories.
    - Example: EPEL (Extra Packages for Enterprise Linux).
  - Local Repositories
    - Stored on a local server or machine.
    - Used in environments with limited or no internet access.

RPM Query

- The rpm command has a subcommand called query (-q) , which is used to query information about installed packages or RPM files.
- Basic Syntax:
  - rpm -q [options] [package_name]
- Check if a package is installed
  - rpm -q package_name
- List all installed packages
  - rpm -qa
- Get detailed information about a package
  - rpm -qi package_name
- List the files provided by an installed package
  - rpm -ql package_name
- Find which package owns a specific file
  - rpm -qf /path/to/file
- Query information from an RPM file
  - rpm -qip package_file.rpm
- List the contents of an RPM file
  - rpm -qlp package_file.rpm

Linux Task Scheduling

Task Scheduling in Linux

- Task scheduling in Linux refers to the ability to automate the execution of tasks (programs, scripts, or commands) at specific times or intervals.
- It allows users to efficiently manage repetitive tasks or perform one-time actions without manual intervention.
- Types of Task Scheduling in Linux
  - One-Time Scheduling
    - This is used for tasks that need to run only once at a specific time.
    - Tools: **at** and **batch**
  - Recurring Scheduling
    - This is used for tasks that need to run periodically (e.g., daily, weekly, monthly).
    - Tools: **cron** and **anacron**

Task Scheduling in Linux cont…

- (Cron Jobs)
  - Purpose: is used for scheduling recurring tasks that need to run at regular intervals (e.g., daily, weekly, monthly). It is ideal for automating routine tasks.
  - Configuration File: /etc/crontab or user-specific crontab.
  - Syntax:
    - *** * * * * command_to_run**
  - Examples:
    - Run a script daily at 2:30 AM
    - **30 2 * * * /path/to/script.sh**
    - Send an email reminder every Friday at 5 PM.
    - **0 17 * * 5 /path/to/email_reminder.sh**
    - Run a command every 5 minutes
    - ***/5 * * * * /path/to/command**
  - Commands:
    - View user crontab
    - **crontab -l**
  - Edit user crontab
    - **crontab -e**
- Anacron
  - Purpose: Schedule recurring tasks like cron but ensures tasks are run even if the system is off during the scheduled time.
  - Configuration File: /etc/anacrontab
  - Use Case: Ideal for laptops or desktops that may not run 24/7.
  - /var/spool/anacron/

Task Scheduling in Linux cont…

- At
  - Purpose: is used for scheduling one-time tasks that run at a specific time in the future. Once the task is executed, it is no longer retained.
  - Usage:
    - echo "command_to_run" | at time
    - Example:
      - echo "reboot" | at 02:30
    - Commands:
      - List scheduled tasks:
        - atq
      - Remove a scheduled task:
        - atrm job_id
- Batch
  - Purpose: Schedule tasks to run when system load is low.
  - Usage:
    - echo "command_to_run" | batch

Linux Log Management

Linux Log Management

- In Red Hat Enterprise Linux (RHEL), logging is primarily handled by the systemd journald and rsyslogd services.
- The logging system collects and stores messages related to system activities, application events, and errors.
- 1. Systemd Journal (Default Logging System)
  - RHEL uses systemd as its init system, and it includes the journal for logging.
  - The journal stores log messages in a binary format, which can be queried and filtered using the journalctl command.
  - By default, the systemd journal logs are stored in memory, so if the system is rebooted, logs are lost unless persistent storage is configured.
  - Usage:
    - Journalctl - Views the complete journal.
    - journalctl -u <service_name> - Views logs for a specific service.
    - journalctl –f - Follows the latest log entries
- Traditional Log Files
  - Alongside systemd logs, RHEL uses traditional text-based log files stored in /var/log/.
  - These are plain text logs generated by various system services and applications. Common log files include:
    - /var/log/messages: General system logs, including kernel messages and other system-level events.
    - /var/log/secure: Logs related to authentication, sudo activity, and security-related events.
    - /var/log/cron: Logs for cron jobs.

Linux Log Management

Understanding Severity Levels of rsyslog:

Level Name	Priority Number	Meaning
emerg	0	System is unusable
alert	1	Action must be taken immediately
crit	2	Critical conditions (serious failures)
err	3	Error conditions
warning	4	Warning messages
notice	5	Normal but significant events
info	6	Informational messages
debug	7	Debugging messages

Linux Disk Partition Management

Undertanding Disk Layout

- What is Partition?
  - A **disk partition** is a logically defined section of a hard drive or SSD that acts as an independent storage unit.
  - Partitions help organize data, separate operating systems, and improve system management.
- Why Partition a Disk?
  - **Multi-OS Setup** – Allows running multiple operating systems (e.g., Windows and Linux).
  - **Data Organization** – Separates system files, applications, and personal data.
  - **Backup & Recovery** – Keeps important data isolated from the OS in case of crashes.
  - **Performance Optimization** – Reduces file fragmentation and improves access speed.

What is partition table?

- A partition table is a data structure stored on a disk that defines how the storage space is divided into partitions.
- It tells the operating system where partitions begin and end, their type, and how they are used.
- Without a partition table, a disk would be considered unallocated and unusable by an OS.
- Types of Partition Tables
  - MBR
  - GPT
- MBR (Master Boot Record)
  - The older partitioning scheme, used since the 1980s.
  - Uses BIOS-based booting.
  - Supports up to 4 primary partitions or 3 primary + 1 extended.
  - Maximum disk size: 2TB.
  - Structure of MBR:
    - Bootloader – Loads the OS.
    - Partition Table (64 bytes) – Holds up to 4 partition entries.

What is partition table cont...

- GPT (GUID Partition Table)
  - The modern partitioning standard, replacing MBR.
  - Supports up to 128 partitions (depending on OS).
  - Works with disks larger than 2TB.
  - Required for UEFI-based systems.
  - Structure of GPT:
    - Partition Table – Holds up to 128 partition entries.
    - Bootloader– Loads the OS.

Feature	MBR (Master Boot Record)	GPT (GUID Partition Table)
Max Partitions	4 primary (or 3 primary + 1 extended)	128 (depending on OS)
Max Disk Size	2TB	9.4 ZB (zettabytes)
Boot Mode	BIOS	UEFI
Redundancy	No redundancy	Backup GPT stored at disk end
Corruption Resistance	Prone to corruption	More resilient

What is a file system?

- A file system is a method used by an operating system to organize, store, retrieve, and manage files on a storage device (such as a hard drive, SSD, or USB). It defines how data is structured, accessed, and secured.
- Common File Systems
  - FAT32 (File Allocation Table 32)
    - Compatible with Windows, Linux, macOS, and most devices (USB drives, SD cards).
    - Simple structure and widely supported.
    - Maximum file size: 4GB.
    - Maximum partition size: 2TB.
  - exFAT (Extended File Allocation Table)
    - Supports larger file sizes (no 4GB limit).
    - Better optimized for flash drives and SD cards.
    - Works on Windows, macOS, and Linux (with extra drivers).
  - NTFS (New Technology File System)
    - Supports large files and partitions (exceeding 2TB).
    - upports file permissions, encryption (BitLocker), and compression.
    - Not fully supported by macOS (read-only without third-party drivers).
    - More complex than FAT32, so slower on small flash drives.

What is a file system cont…

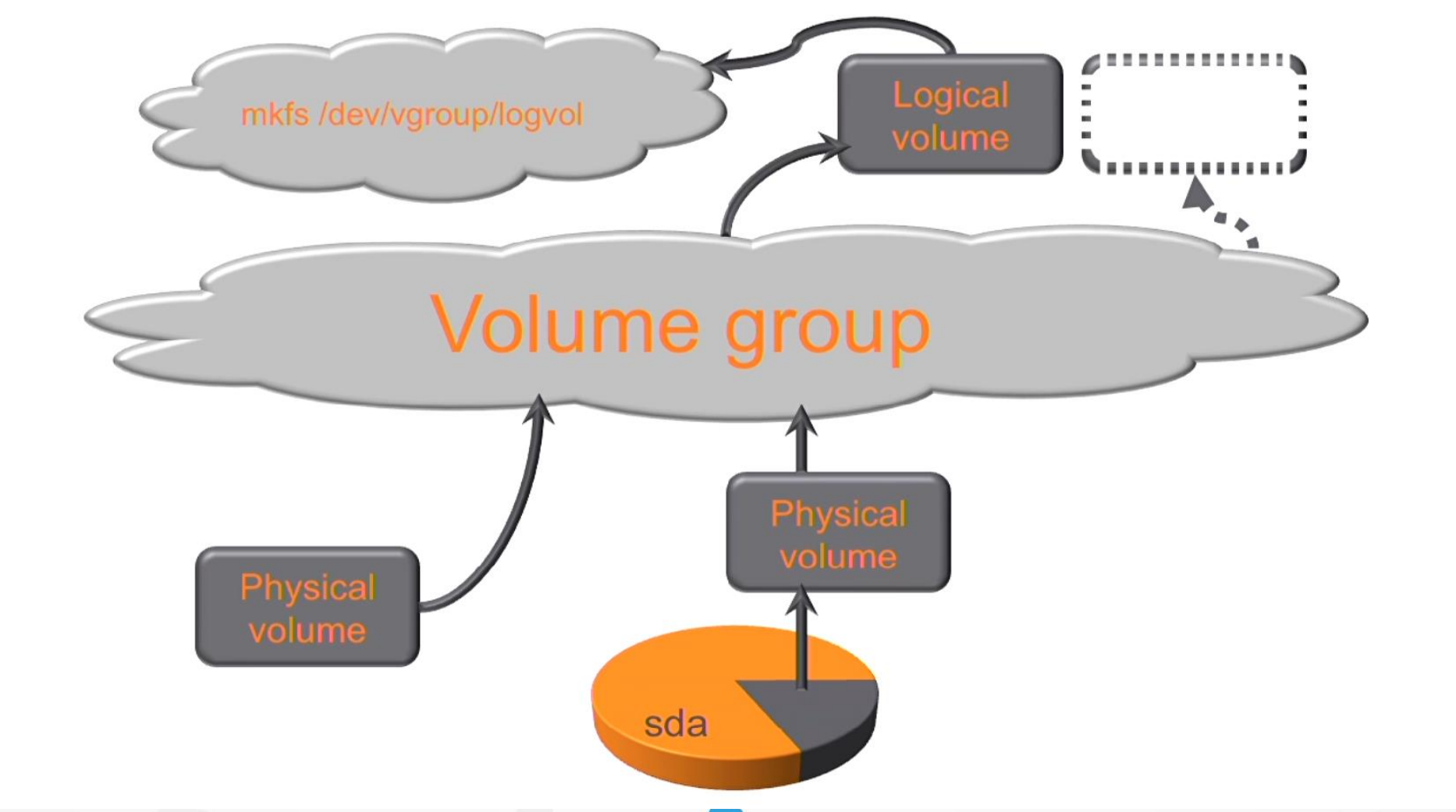
- Common File Systems
  - ext4 (Fourth Extended Filesystem)
    - Default file system for Linux distributions.
    - Handles large files and partitions efficiently.
    - Not natively supported on Windows.
    - Slower than XFS for handling large files.
  - XFS (High-Performance Journaled File System)
    - Optimized for high-performance and large files.
    - Scales well for servers and enterprise use.
    - Uses journaling for data protection.
    - Not easily resizable like ext4.
    - Not natively supported on Windows.
- Which File System Should You Use?
  - Windows: NTFS for internal drives, exFAT for USB drives.
  - Linux: ext4 for general use, XFS for servers.
  - macOS: APFS for SSDs, HFS+ for older systems.
  - Cross-Platform: exFAT for compatibility between Windows, macOS, and Linux.

Managing Linux LVM Logical Volumes

Managing Linux LVM Logical Volumes

- LVM (Logical Volume Manager) is a storage management system in Linux that provides flexible disk space management.
- Instead of directly partitioning a disk into fixed-size partitions, LVM allows you to create and manage logical volumes dynamically.
- Advantages of LVM:
  - Flexible resizing – You can resize logical volumes dynamically.
  - Easier disk management – Multiple disks can be combined into a single volume.
  - Striping and mirroring – You can distribute data across multiple physical volumes for performance (striping) or redundancy (mirroring).
  - Snapshot support – You can create snapshots of volumes for backup or testing purposes.
- Key Concepts of LVM:
  - Physical Volume (PV) – This refers to the actual physical storage devices (HDDs, SSDs, or partitions) that are initialized for use with LVM.
  - Volume Group (VG) – A collection of one or more physical volumes that form a storage pool.
  - Logical Volume (LV) – Equivalent to a partition, but more flexible. You can resize or move it easily without affecting the underlying storage.

Managing Linux LVM Logical Volumes



What is Device Mapper?

- The Device Mapper is a kernel-based framework in Linux that allows flexible management of block devices.
- It is used for creating logical volumes, RAID configurations, disk encryption (LUKS), and snapshots. The LVM (Logical Volume Manager) relies on the Device Mapper to manage logical volumes (LVs).

Linux Systemd

What is sytemd?

- **Systemd** is a system and service manager for Linux, responsible for managing the system startup, running services, and handling system resources. It replaces the older **SysVinit** and **Upstart** systems.
- Systemd Components
  - Systemd works by managing different types of **units**, which are configuration files stored in /etc/systemd/system/ and /usr/lib/systemd/system/

Unit Type	Description	Example File
Service	Manages system services	nginx.service
Target	Group of units (like runlevels)	multi-user.target
Socket	Manages socket activation	cups.socket
Mount	Controls filesystem mounting	home.mount
Timer	Runs tasks at scheduled times (like cron)	backup.timer

What is targets?

- Targets are a way to group units (services, sockets, mounts, etc.) and manage system states. Targets replace traditional runlevels from SysVinit.
- System State Targets
  - These targets define the operational state of the system.

Target Name	Description
multi-user.target	Multi-user mode without a graphical interface (like runlevel 3 in SysVinit).
graphical.target	Multi-user mode with a graphical interface (like runlevel 5).
rescue.target	Single-user mode, with only essential system services.
emergency.target	Similar to rescue.target, but does not mount filesystems—used for emergency troubleshooting.

- Other Targets:

poweroff.target	Shuts down the system.
reboot.target	Reboots the system.
bluetooth.target	Starts Bluetooth services.
printer.target	Initializes printer services.
sound.target	Manages sound system dependencies.
network.target	Indicates that networking is available.

How to reset root password?

- Step 1: Reboot the System
  - Restart your RHEL system.
  - During boot, press e when the GRUB menu appears.
- Step 2: Edit the GRUB Entry
  - Find the line that starts with linux or linux16.
  - At the end of this line, add:
    - rd.break
  - Press Ctrl + X to boot with these changes.
- Step 3: Remount Root Filesystem
  - mount -o remount,rw /sysroot
- Step 4: Change the root environment
  - chroot /sysroot
- Step 5: Reset the Root Password
  - passwd root
- Step 6: Restore SELinux Context
  - touch /.autorelabel
- EXIT

Linux Firewall Management

What is linux firewall?

- A firewall is a security system that controls incoming and outgoing network traffic based on predefined rules.
- Firewalls help filter traffic, block unauthorized access, and allow legitimate communications.
- In linux Netfilter is the core packet filtering framework in the Linux kernel.
- Linux provides multiple firewall solutions, including:
  - iptables (Older, manual rules management)
  - nftables (Modern replacement for iptables)
  - firewalld (Dynamic firewall that simplifies rule management)
  - ufw (Uncomplicated Firewall, mainly for Ubuntu)
- What is firewalld?
  - Firewalld is a dynamic firewall management tool for Linux, replacing traditional static firewall tools like iptables.
  - It provides runtime and permanent configurations, making firewall rules more flexible.
- Basic firewalld commands:
  - Check if Firewalld is Running
    - systemctl status firewalld
  - View Active Firewall Rules:
    - firewall-cmd --list-all
  - List Available Zones
    - firewall-cmd --get-zones

SELinux

What is SELinux?

- Security-Enhanced Linux (SELinux) is a security system which is built into the Linux kernel.
- It adds an extra layer of security by defining policies that strictly control what processes can access files, directories, ports, and other system resources.
- How SELinux Rules Work
  - SELinux (Security-Enhanced Linux) enforces security policies through a system of rules that control access to files, processes, and system resources.
  - These rules are defined in SELinux policies and determine whether a particular action is allowed or denied based on security contexts.
- SELinux Policy Model
  - SELinux follows a Mandatory Access Control (MAC) model, meaning access control is enforced by system policies rather than discretionary user permissions (like traditional Unix file permissions).
  - SELinux policies define rules based on:
    - **Subjects** – Processes or users that request access.
    - **Objects** – Files, directories, ports, or resources being accessed.
    - **Actions (Permissions)** – What the subject wants to do with the object (read, write, execute, etc.).
  - If a process tries to access an object, SELinux checks its rules to decide whether to allow or deny the action.

What is SELinux?

- SELinux Security Contexts
  - Whenever a process (subject) tries to access a file, port, or other resources (object), SELinux checks their security contexts. If the rules in the SELinux policy allow the action, access is granted; otherwise, it is denied.
  - Every object and subject in the system has a security context that includes:
    - **user:role:type:level**
    - **User**: SELinux user (e.g., system_u, unconfined_u)
    - **Role**: SELinux role (e.g., object_r, sysadm_r)
    - **Type:** Defines access permissions (e.g., httpd_t, httpd_sys_content_t)
    - **Level** (optional): Used for Multi-Level Security (MLS)
- To check security contexts:
  - ls -Z /var/www/html/index.html
  - ps auxZ | grep httpd

What is SELinux?

- SELinux Modes:
  - SELinux operates in three different modes:
    - **Enabled:**
      - **Enforcing** – SELinux policies are enforced, and unauthorized access is denied.
      - **Permissive** – SELinux policies are not enforced but logged. This is useful for troubleshooting.
    - **Disabled** – SELinux is turned off completely.
  - To check the current mode:
    - sestatus
    - getenforce
  - To temporarily change the mode (without rebooting):
    - setenforce 0
    - setenforce 1
  - To permanently change the mode, edit the **SELinux configuration file**:
    - vi /etc/selinux/config
    - reboot

What is SELinux?

- Changing File/Directory SELinux Context Type
  - To change the SELinux **type** of a file, you can use **chcon** (temporary) or **semanage + restorecon** (permanent).
  - Temporary Change (chcon)
    - The chcon command **temporarily** changes the SELinux type but does not survive a system relabel or restorecon
      - chcon -t <new_type> <file>
  - Permanent Change (semanage fcontext)
    - If you want the change to **persist** across reboots and relabels, use **semanage** and then restore the context with restorecon:
      - semanage fcontext -a -t httpd_sys_content_t "/var/www/html(/.*)?"
      - restorecon -Rv /var/www/html/

Basics of Shell Scripting

What is Shell Script?

- A **shell script** is a text file containing a series of commands executed sequentially by the shell.
- Creating a Simple Shell Script
  - Use a text editor (e.g., vi, nano).
  - Begin the script with a shebang (#!/bin/bash), which tells the system to use Bash.
  - Save the file with a .sh extension (not mandatory but recommended).
  - Give it execute permissions (chmod +x script.sh).
  - Run the script (./script.sh).

What are Variables?

- Variables in shell scripting are used to store and manipulate data, similar to other programming languages. They help in dynamic scripting by allowing values to be reused and modified.
- Declaring a Variable
  - #!/bin/bash
  - name="Sam"
  - echo "Hello, $name!“
- Variable Types
  - User-Defined Variables
    - Set by the user within the script.
    - #!/bin/bash
    - greeting="Welcome"
    - echo "$greeting to shell scripting!"
  - System Variables
    - #!/bin/bash
    - echo "User: $USER"
    - echo "Home Directory: $HOME"
    - echo "Current Shell: $SHELL"
    - echo "Present Working Directory: $PWD"

What are Conditional Statements?

- Conditional statements allow your script to make decisions based on conditions. Bash supports different types of conditions, such as **if-else and logical operators**.
- If-Else Statement
  - The if statement checks if a condition is true and executes commands accordingly.
    - if [condition]; then
    - # Commands if condition is true
    - else
    - # Commands if condition is false
    - fi
- Comparison Operators
  - Numeric Comparisons

Operator	Meaning
-eq	Equal to
-ne	Not equal to
-gt	Greater than
-lt	Less than
-ge	Greater than or equal to
-le	Less than or equal to

- String Comparisons

Operator	Meaning
=	Equal
!=	Not equal
-z	String is empty
-n	String is not empty

What are Conditional Statements?

- File Conditionals
  - **Operator** **Meaning**
  - -f File exists and is a regular file
  - -d Directory exists
  - -e File or directory exists
  - -r File is readable
  - -w File is writable
  - -x File is executable
- Example:

```
#!/bin/bash
file="/etc/passwd"

if [ -f $file ]; then
   echo "File $file exists."
else
   echo "File does not exist."
fi
```

What are Loops?

- A **loop** in shell scripting is used to repeatedly execute a block of code until a specific condition is met.
- Loops are essential for automating repetitive tasks, such as processing files, iterating through lists, or executing commands multiple times.
- Types of Loops:
  - For loop
  - While loop
- For Loop
  - Used when you need to iterate over a list of items.
    - for i in 1 2 3 4 5
    - do
    - echo "Iteration $i"
    - done
- While Loop
  - Runs as long as the condition is **true**.
    - count=1
    - while [$count -le 5]
    - do
    - echo "Count: $count"
    - ((count++))
    - done

Basics Container Management

What is a container?

- A **container** is a lightweight, portable unit that encapsulates an application and its dependencies (like libraries, configurations, and environment variables) into a single package.
- The key idea behind containers is **isolation**: they allow an application to run in any environment consistently without worrying about conflicts with the host system or other applications.
- Key Characteristics of Containers:
  - Isolation
  - Lightweight
  - Portability
  - Consistency
  - Fast Start-Up
- How Containers Work:
  - **Images**: Containers are created from **images**, which are read-only templates. These images include everything the application needs to run (e.g., the application code, libraries, and dependencies).
  - **Runtime**: Once the container is created, the container runtime (e.g., Docker, Podman) manages its execution and ensures it runs in isolation.
  - **Namespaces**: Containers use namespaces (like **PID**, **Network**, and **Filesystem** namespaces) to provide isolation from other containers and the host system.

What is a Container Runtime?

- A **container runtime** is software responsible for **running containers** on a system.
- It provides the necessary functionalities to create, start, stop, and manage containerized applications by isolating them from the host system.
- Types of Container Runtimes
  - Container runtimes are classified into two categories:
  - 1. Low-Level (OCI) Runtimes
    - These runtimes handle the actual execution of containers.
    - Examples:
      - runc (used by Docker, Podman, Kubernetes)
      - crun (lightweight alternative to runc)
      - Kata Containers (provides additional security by using virtual machines)
  - 2. High-Level (Container Management) Runtimes
    - These runtimes provide **higher-level tools** to manage containers using low-level runtimes. Examples:
      - Docker Engine (uses runc internally)
      - Podman (uses runc or crun)
      - containerd (used by Kubernetes, managed by Docker)
      - CRI-O (lightweight runtime for Kubernetes)

END